Ders sunumu · Türkçe çeviri

three.js Kılavuzu

Kurulum, sahne oluşturma, temeller, temel geometriler ve sahne grafiği



Giriş

Sunum planı

01

Kurulum

NPM veya CDN ile projeyi hazırlamak

02

Sahne oluşturma

Sahne, kamera, render ve dönen küp

03

Temeller

Renderer, Mesh, ışık ve modüller

04

Temel geometriler

Primitives, TextGeometry ve Points

05

Sahne grafiği

Hiyerarşi, yerel alan, tank örneği

06

Özet

Mimari ve kurulum adımları

01

Kurulum

Installation · Projeyi hazırlamak ve çalıştırmak

Proje yapısı (Project structure)

Her three.js projesinin bir HTML dosyasına ve three.js kodunu çalıştıran bir JavaScript dosyasına ihtiyacı vardır. İsimler zorunlu değildir; rehber boyunca tutarlılık için kullanılır.

`index.html`

Web sayfasını tanımlar ve main.js dosyasını modül olarak yükler.

`main.js`

three.js kodunuz burada çalışır; en üstte three içe aktarılır.

`public/`

“Static” klasör: dokular, ses ve 3B modeller buraya konur; dosyalar değiştirilmeden yayınlanır.

01 · Kurulum · Çoğu kullanıcı için önerilen yol

Seçenek 1: NPM ve derleme aracı • Geliştirme

1. Node.js'i kurun.
2. Proje klasöründe three.js'i ve derleme aracı Vite'i kurun.
3. Terminalde npx vite çalıştırın.
4. Terminaldeki adresi (örn. localhost:5173) tarayıcıda açın.

```
# three.js
npm install --save three

# vite
npm install --save-dev vite

# sunucuyu başlat
npx vite
```

Vite yalnızca geliştirme sırasında kullanılır, nihai web sayfasının parçası değildir. Sayfa boş olacak: sahne oluşturmaya hazırsınız.

Seçenek 1 • Yayın (Production)

Web uygulaması yayına hazır olduğunda Vite'a üretim derlemesi çalıştırmasını söylemek yeterlidir.

```
npx vite build
```

Derlenir
Uygulamanın kullandığı her şey.

Optimize edilir
Yayın için hazırlanır.

dist/ klasörü
İçeriği sitenizde barındırmaya hazırdır.

Seçenek 2: CDN’den içe aktarma • Import map

main.js içindeki 'three' bir npm paketidir; tarayıcı bunun ne olduğunu bilmez. Paketin nereden alınacağını index.html içindeki import map söyler (head etiketinde, stillerden sonra).

```
<script type="importmap">
{
  "imports": {
    "three": "https://cdn.jsdelivr.net/npm/three@<version>/build/three.module.js",
    "three/addons/": "https://cdn.jsdelivr.net/npm/three@<version>/examples/jsm/"
  }
}
</script>
```

version yerine gerçek bir sürüm yazın, örneğin v0.149.0. En güncel sürüm npm sürüm listesinde bulunur.

Seçenek 2 • Yerel sunucu çalıştırma

HTML dosyasına çift tıklamak teknik olarak mümkündür; ancak ileride kullanacağımız önemli özellikler, güvenlik nedenleriyle bu şekilde çalışmaz.

1. Node.js'i kurun.
2. Proje dizininde serve'i çalıştırın.
3. Terminaldeki adresi (localhost:3000) açın.

```
npx serve .
# → http://localhost:3000
```

Sayfa boş olacak: sahne oluşturmaya hazırsınız.

Daha fazla yerel sunucu (More local servers)

Komut satırı

```
npx http-server
npx five-server
python -m http.server
python -m SimpleHTTPServer
php -S localhost:8000
```

Node.js, Python 3.x, Python 2.x, PHP 5.4+ (önce kurulmalı)

Grafik arayüz (GUI)

Bilgisayarınızda bir uygulama penceresi olarak çalışır.

Servez

Editör eklentileri

İstek üzerine basit bir sunucu başlatır.

VS Code: Five Server
VS Code: Live Server
Atom: Live Server

Hepsi temelde aynı şekilde çalışır: dosyalarınızı bir adres üzerinden tarayıcıya sunar.

Seçenek 2 • Yayın (Production)

Kaynak dosyalarınızı barındırma sağlayıcınıza gönderin; hiçbir şeyi derlemeye gerek yok.

Takasın bedeli: bakım

Import map'i tüm bağımlılıklarla (ve onların bağımlılıklarıyla) güncel tutmanız gerekir.

Takasın bedeli: CDN'e bağımlılık

Bağımlılıkları barındıran CDN geçici olarak kapanırsa siteniz de çalışmayı durdurur.

ÖNEMLİ

Tüm bağımlılıkları three.js'in aynı sürümünden ve aynı CDN'den içe aktarın. Farklı kaynaklardan dosya karıştırmak kodun mükerrer eklenmesine ya da beklenmedik hatalara yol açabilir.

Eklentiler (Addons)

Kontroller, yükleyiciler ve son işleme efektleri addons/ dizinindedir. Ayrıca kurulmaları gerekmez, ancak ayrıca içe aktarılmaları gerekir.

```
import * as THREE from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';

const controls = new OrbitControls( camera, renderer.domElement );
const loader = new GLTFLoader();
```

Üçüncü taraf three.js projeleri ise ayrıca kurulmalıdır (Libraries and Plugins sayfası).

01 · Kurulum · Sonraki adımlar (Next Steps)

Artık bir sahne oluşturmaya hazırsınız.

Proje yapısı hazır, sunucu çalışıyor, sayfa boş.

02

Sahne oluşturma

Creating a scene · Dönen bir küp ile ilk uygulama

Başlamadan önce (Before we start)

Kurulum kılavuzunu incelediğinizi varsayıyoruz. Hazır olması gerekenler:

- ✓ Aynı proje yapısı: index.html ve main.js
- ✓ three.js yüklü
- ✓ Bir derleme aracı çalışıyor ya da CDN ve import map ile yerel sunucu kullanılıyor

Takılırsanız: kılavuzun sonunda çalışan bir örnek var.

Sahneyi oluşturma (Creating the scene)

Bir şey görüntülemek için üç şey gerekir: **sahne** (scene), **kamera** (camera) ve **render edici** (renderer).

```
import * as THREE from 'three';

const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(
  75, window.innerWidth / window.innerHeight, 0.1, 1000 );

const renderer = new THREE.WebGLRenderer();
renderer.setSize( window.innerWidth, window.innerHeight );
document.body.appendChild( renderer.domElement );
```

PerspectiveCamera ve render edici boyutu

75	en / boy	0.1	1000
Görüş alanı (FOV), derece cinsinden.	**En-boy oranı:** yanlışsa görüntü sıkışır.	**near:** bundan yakındakiler çizilmez.	**far:** bundan uzaktakiler çizilmez.

Render edicinin boyutu: performans için daha küçük değer verilebilir. Üçüncü argüman false ise canvas boyutu korunur, çözünürlük yarıya iner.

```
renderer.setSize( window.innerWidth/2, window.innerHeight/2, false );
```

02 · Sahne oluşturma

Küpü ekleme

```
const geometry = new THREE.BoxGeometry( 1, 1, 1 );
const material = new THREE.MeshBasicMaterial( { color: 0x00ff00 } );
const cube = new THREE.Mesh( geometry, material );
scene.add( cube );

camera.position.z = 5;
```

Geometry

Küpün tüm noktalarını ve yüzlerini içerir.

Material

Rengi verir; 0x00ff00 yeşildir (hex renk).

Mesh

İkisini birleştirir; sahneye eklenir ve hareket ettirilir.

Kamera

Her şey (0,0,0)'a eklenir; kamerayı geri alırız.

Sahneyi render etme (Rendering the scene)

Kodu kopyalarsanız hâlâ hiçbir şey görmezsiniz: henüz gerçekten bir şey çizmiyoruz. Bunun için bir animasyon (render) döngüsü gerekir.

```
function animate( time ) {
  renderer.render( scene, camera );
}
renderer.setAnimationLoop( animate );
```

- Ekran her yenilendiğinde sahne çizilir (tipik olarak saniyede 60 kez).
- Arka planda requestAnimationFrame kullanılır; setInterval'dan üstündür.
- Kullanıcı başka sekmeye geçince durur: işlem gücü ve pil boşa harcanmaz.

Küpü canlandırma (Animating the cube)

Artık yeşil bir kutu görmelisiniz. Onu döndürerek biraz daha ilginç yapalım: kodu renderer.render çağrısının hemen üstüne ekleyin.

```
function animate( time ) {

  cube.rotation.x = time / 2000;
  cube.rotation.y = time / 1000;

  renderer.render( scene, camera );
}
```

Uygulama çalışırken hareket etmesini ya da değişmesini istediğiniz her şey animasyon döngüsünden geçer.

Sonuç (The result): ilk three.js uygulamanız

1. Sahne, kamera ve render edici
2. Geometri ve malzeme
3. Mesh'i sahneye ekle
4. Kamerayı geri al (z = 5)
5. animate döngüsünde döndür

Tebrikler! Basit, ama bir yerden başlamak gerekir.

Kodun tamamı kılavuzda düzenlenebilir bir canlı örnek (jsfiddle) olarak var: üzerinde oynayarak nasıl çalıştığını anlayın.

03

Temeller

Fundamentals · three.js'in yapı taşları ve "Hello Cube!"

three.js ve WebGL

three.js, 3B içeriği web sayfasına mümkün olduğunca kolay getirmeye çalışan bir 3B kütüphanedir. Çoğu zaman (ama her zaman değil) 3B çizmek için WebGL kullanır.

WebGL

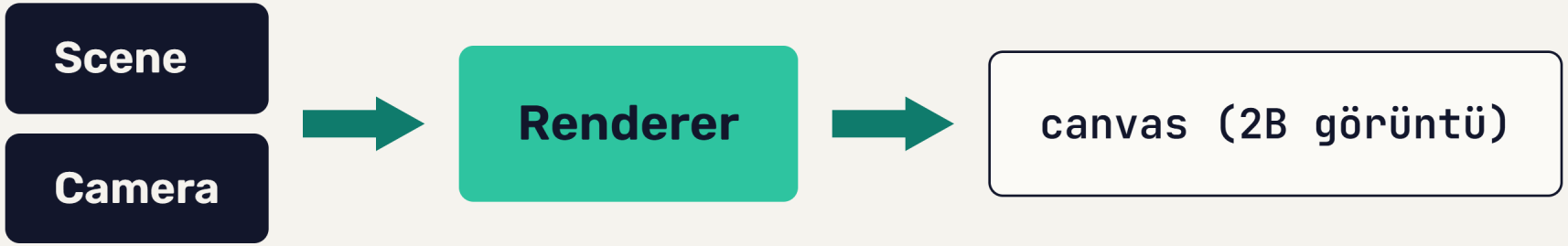
Yalnızca noktaları, çizgileri ve üçgenleri çizen çok düşük seviyeli bir sistem. Yararlı bir şey için epeyce kod gerekir.

three.js

Sahneler, ışıklar, gölgeler, malzemeler, dokular ve 3B matematik: WebGL'de kendiniz yazacağınız her şeyi o halleder.

Önkoşul: JavaScript bilgisi (çoğunlukla ES6 stili). Eski tarayıcılar için Babel gibi bir transpiler kullanılabilir.

Renderer ve sahne grafiği (scenegraph)



Renderer

Three.js'in ana nesnesi. Sahneyi ve kamerayı alır; kamera görüş hacmindeki (frustum) 3B bölümü 2B görüntü olarak çizer.

Sahne grafiği

Ağaç yapısı: kökte Scene; altında Mesh, Light, Group, Object3D ve Camera. Çocuklar ebeveynine göre konumlanır.

Mesh, Geometry, Material, Texture, Light

Mesh = Geometry + Material + konum, yönelim, ölçek

Konum, yönelim ve ölçek ebeveynine göredir.

Geometry

Şeklin köşe noktası verisi: küre, küp, düzlem, ağaç, bina...

Material

Yüzey özellikleri: renk, parlaklık; Texture'lara başvurabilir.

Texture

Dosyadan, canvas'tan ya da başka bir sahneden gelen görüntü.

Light

Farklı türde ışıkları temsil eder.

Paylaşım: iki mavi küp = iki Mesh (konumlar için), ama tek Geometry ve tek Material yeterlidir.

three.js'i yükleme ve canvas

```
<canvas id="c"></canvas>

<script type="module">
import * as THREE from 'three';
const canvas = document.querySelector('#c');
const renderer = new THREE.WebGLRenderer({antialias: true, canvas});
</script>
```

type="module"

import ile yüklemek için şarttır; r147'den beri tek doğru yol budur.

Canvas'ı biz veriyoruz

Canvas'ı herhangi bir yere koyarız, kod onu bulur: daha esnektir.

Kamera ve frustum

```
const fov = 75;
const aspect = 2;  // canvas varsayılanı
const near = 0.1;
const far = 5;
const camera = new THREE.PerspectiveCamera(fov, aspect, near, far);
camera.position.z = 2;
```

fov = 75°

Dikey görüş alanı. Açılar genelde radyandır; bu kamera derece alır.

aspect = 2

Canvas 300×150 piksel: 300/150 = 2.

Frustum

near ve far arasındaki, tepesi kesik piramit. İçindeki çizilir, dışındaki çizilmez.

z = 2

Kamera −Z yönüne bakar, +Y yukarıdır; küp başlangıçta, kamera geridedir.

Sahne, geometri, malzeme ve Mesh

```
const scene = new THREE.Scene();

const geometry = new THREE.BoxGeometry(1, 1, 1);
const material = new THREE.MeshBasicMaterial({color: 0x44aa88});
const cube = new THREE.Mesh(geometry, material);
scene.add(cube);
```

Scene

Sahne grafiğinin kökü; çizilecek her şey eklenir.

BoxGeometry

Genişlik, yükseklik, derinlik; köşe noktaları.

Material

Renk, 6 haneli hex değer (CSS stili).

Mesh

Geometry + Material + ebeveynine göre konum.

Render döngüsü ve animasyon

```
function render(time) {
  time *= 0.001;  // saniyeye çevir

  cube.rotation.x = time;
  cube.rotation.y = time;

  renderer.render(scene, camera);
  requestAnimationFrame(render);
}
requestAnimationFrame(render);
```

- requestAnimationFrame, tarayıcıya “bir şeyi canlandırmak istiyorum” demektir.
- Fonksiyona sayfa yüklendiğinden beri geçen süre (milisaniye) gelir.
- Dönüş radyandır: 2π ≈ 6,28 saniyede bir tam tur.
- Döngüyü başlatmak için dışarıda bir kez çağrılır.

Işık ekleme: DirectionalLight

```
const color = 0xFFFFFF;
const intensity = 3;
const light = new THREE.DirectionalLight(color, intensity);
light.position.set(-1, 2, 4);
scene.add(light);
```

Konum ve hedef

Konum (−1, 2, 4): kameranın solunda, üstünde, arkasında. Hedef (0, 0, 0).

MeshBasicMaterial

Işıklardan etkilenmez.

MeshPhongMaterial

Işıktan etkilenir; küp artık açıkça 3B görünür.

Üç küp: aynı geometri, farklı malzeme

```
function makeInstance(geometry, color, x) {
  const material = new THREE.MeshPhongMaterial({color});
  const cube = new THREE.Mesh(geometry, material);
  scene.add(cube);
  cube.position.x = x;
  return cube;
}
const cubes = [
  makeInstance(geometry, 0x44aa88,  0),
  makeInstance(geometry, 0x8844aa, -2),
  makeInstance(geometry, 0xaa8844,  2),
];
```

Mesh · x = 0 · 0x44aa88

Mesh · x = −2 · 0x8844aa

Mesh · x = +2 · 0xaa8844

3 Mesh, 1 BoxGeometry, 3 MeshPhongMaterial. Her küp biraz farklı hızda döner; kenardaki küpler frustum'un dışına taşar ve çarpık görünür.

ES6 modülleri, three.js ve klasör yapısı

```
<script type="importmap">
{
  "imports": {
    "three": "./path/to/three.module.js",
    "three/addons/": "./different/path/to/examples/jsm/"
  }
}
</script>

<script type="module">
import * as THREE from 'three';
import {OrbitControls} from
'three/addons/controls/OrbitControls.js';
</script>
```

- r147'den beri tercih edilen yol: ES6 modülleri ve import map.
- Import map olmadan 'three' büyük olasılıkla hata verir.
- Yollar yalnızca ./ veya ../ ile başlar.
- Eklentiler de import map'e eklenir. CDN de kullanılabilir.

04

Temel geometriler

Primitives · Parametrelerle üretilen 3B şekiller

Temel geometriler (Primitives) ve yardımcılar

Primitives nedir?

Çalışma zamanında parametrelerle oluşturulan 3B şekiller: dünya küresi için küre, 3B grafik için kutular. Çoğu parametrenin varsayılan değeri vardır.

Ne zaman kullanılır?

3B'ye ilk adımlarda ve deneylerde. Çoğu uygulamada ise Blender, Maya veya Cinema 4D'de hazırlanmış modeller yüklenir.

`EdgesGeometry`

Başka bir geometriyi alır; yüzeyler arasındaki açı eşik değerini (thresholdAngle) aşınca kenar üretir. Yüzlerin ortasındaki üçgen çizgilerini kaldırır.

`WireframeGeometry`

Her kenar için tek bir çizgi parçası (2 nokta) üretir. Tek üçgen: 3 nokta yerine 6 nokta ve 3 çizgi parçası.

Hazırlık: arka plan, kamera ve addObject

Açık gri arka plan; kamera tüm nesneleri görecek şekilde uzaklaşır.

```
scene.background =
  new THREE.Color(0xAAAAAA);

const fov = 40;     // 75 idi
const far = 1000;   // 5 idi
camera.position.z = 120;
// 2 idi
```

addObject: x, y konumunu ayarlar, sahneye ve diziye ekler.

```
const objects = [];
const spread = 15;

function addObject(x, y, obj) {
  obj.position.x = x * spread;
  obj.position.y = y * spread;
  scene.add(obj);
  objects.push(obj);
}
```

Rastgele renkli malzeme: createMaterial

```
function createMaterial() {
  const material = new THREE.MeshPhongMaterial({
    side: THREE.DoubleSide,
  });

  const hue = Math.random();
  const saturation = 1;
  const luminance = .5;
  material.color.setHSL(hue, saturation,
luminance);

  return material;
}
```

HSL renk

Ton 0–1 (kırmızı 0, yeşil 0.33, mavi 0.66). Doygunluk 0–1 (0 gri). Parlaklık 0–1 (0 siyah, 0.5 tam renk, 1 beyaz).

side: DoubleSide

Üçgenlerin iki yüzü de çizilir. PlaneGeometry ve ShapeGeometry gibi 2B şekillerin arka yüzü aksi hâlde kaybolur. Daha yavaştır; yalnızca gerekene uygulayın.

addSolidGeometry ve ilk örnek: kutu

```
function addSolidGeometry(x, y, geometry) {
  const mesh = new THREE.Mesh(geometry, createMaterial());
  addObject(x, y, mesh);
}

{
  const width = 8;
  const height = 8;
  const depth = 8;
  addSolidGeometry(-2, -2, new THREE.BoxGeometry(width, height, depth));
}
```

Temel geometrilerin çoğu bu kalıpla eklenir. İstisnalar:

TextGeometry **EdgesGeometry** **WireframeGeometry** **Points**

TextGeometry: yazı tipini yükleme

```
const loader = new FontLoader();

function loadFont(url) {
  return new Promise((resolve, reject) ⇒ {
    loader.load(url, resolve, undefined, reject);
  });
}

async function doit() {
  const font = await loadFont(url);
  const geometry = new TextGeometry('three.js', {font, size: 3.0});
  ...
}
```

- Mesh için önce 3B yazı tipi verisi yüklenmelidir.
- Yükleme eşzamansızdır (asynchronous): beklemeden geometri kurulamaz.
- loadFont, yüklemeyi Promise'e çevirir.
- async doit() içinde await ile font gelir.

Metni kendi merkezi etrafında döndürme

Varsayılan olarak metnin dönme merkezi sol kenarındadır. Düzeltmek için merkezi ofset olarak alır, sonra bir Object3D ebeveyni kullanırız.

```
geometry.computeBoundingBox();
geometry.boundingBox.getCenter(mesh.position).multiplyScalar(-1);

const parent = new THREE.Object3D();
parent.add(mesh);

addObject(-1, -1, parent);
```

1. Sınırlayıcı kutuyu (bounding box) hesapla.

2. Merkezi al, -1 ile çarp: metin ortaya kayar.

3. Mesh'i Object3D'nin çocuğu yap; ofset korunur.

Çizgi tabanlı geometriler: addLineGeometry

EdgesGeometry ve WireframeGeometry, addSolidGeometry yerine addLineGeometry çağırır.

```
function addLineGeometry(x, y, geometry) {
  const material = new THREE.LineBasicMaterial({color: 0x000000});
  const mesh = new THREE.LineSegments(geometry, material);
  addObject(x, y, mesh);
}
```

LineBasicMaterial

Siyah çizgi malzemesi.

LineSegments

Mesh'in sarıcısı: her parça için 2 nokta ile çizgi çizdiğinizi three.js'e bildirir.

Points ve PointsMaterial

```
// yarıçap 7, 12 yatay, 8 dikey bölme
const geometry = new THREE.SphereGeometry(7, 12, 8);
const material = new THREE.PointsMaterial({
  color: 'red',
  size: 0.2,     // dünya birimi
});
const points = new THREE.Points(geometry, material);
scene.add(points);
```

- Points, LineSegments gibi BufferGeometry alır; ama her köşede (vertex) bir nokta çizer.
- PointsMaterial'daki size, noktanın büyüklüğünü belirler.
- sizeAttenuation: false yapılırsa nokta, kameraya uzaklıktan bağımsız aynı boyutta kalır (size piksel olur).

Bölme (subdivision) ayarları

Neredeyse tüm şekiller, kaç parçaya bölüneceğini belirleyen ayarlar taşır.

`widthSegments`
Kürenin çevresinde kaç bölme yapılacağı.

`heightSegments`
Yukarıdan aşağıya kaç bölme yapılacağı.

Parametrelerin tamamı için her şeklin dokümantasyon sayfasına bakmak en doğrusudur.

05

Sahne grafiği

Scene Graph · Hiyerarşi, yerel alan ve ebeveyn–çocuk ilişkisi

Sahne grafiği nedir?

Bir 3B motorunda sahne grafiği, düğümlerin (nodes) hiyerarşisidir. **Her düğüm bir yerel alanı (local space) temsil eder.**



Siz de Dünya'da yürürken Dünya'nın hareketini düşünmezsiniz: saatte yaklaşık 1600 km ile dönerken ve Güneş çevresinde yaklaşık 107.000 km/sa ile giderken bile Dünya'nın yerel alanında yaşarsınız.

Güneş: ilk düğüm

```
const sphereGeometry = new THREE.SphereGeometry(1, 6, 6);
const sunMaterial = new THREE.MeshPhongMaterial({emissive:
0xFFFF00});
const sunMesh = new THREE.Mesh(sphereGeometry, sunMaterial);
sunMesh.scale.set(5, 5, 5);
scene.add(sunMesh);
objects.push(sunMesh);

camera.position.set(0, 50, 0);
camera.up.set(0, 0, 1);
camera.lookAt(0, 0, 0);
```

- Tek bir küre geometrisi her şey için yeniden kullanılır.
- Yalnızca 6 alt bölme: dönüşü görmek kolay.
- scale 5: Güneş büyük.
- emissive sarı: ışık vurmasa da çizilen renk.
- Merkezde bir nokta ışık (PointLight); kamera tepeden bakar.

Dünya’yı eklemek: ölçek sorunu ve çözümü

Sorun

Dünya’yı sunMesh’in çocuğu yaparsak, Güneş’in yerel alanı 5 kat büyük olduğundan Dünya da 5 kat büyür ve uzaklığı (position.x = 10) 5 katına çıkar.

Çözüm: boş bir düğüm

Geometrisi ve malzemesi olmayan bir Object3D ekleyin; Güneş ve Dünya onun çocukları olsun.

```
const solarSystem = new THREE.Object3D();
scene.add(solarSystem);
solarSystem.add(sunMesh);
solarSystem.add(earthMesh);
```

Object3D, Mesh gibi bir sahne grafiği düğümüdür ama yalnızca bir yerel alanı temsil eder.

Ay ve yörünge düğümleri

```
scene
└ solarSystem
   ├ sunMesh
   └ earthOrbit  x = 10
      ├ earthMesh
      └ moonOrbit  x = 2
         └ moonMesh  ölçek 0.5
```

- **earthOrbit:** Dünya ve Ay'ın ortak yerel alanı; Güneş'ten 10 birim uzakta.
- **moonOrbit:** Ay'ın Dünya'ya uzaklığı (2 birim).
- Her düğüm kendi yerel alanında döner; Ay'ın hesabı Dünya'dan bağımsızdır.

Düğümleri görselleştirme: yardımcılar

`AxesHelper`

Yerel X, Y ve Z eksenlerini 3 çizgiyle gösterir.

`GridHelper`

X,Z düzleminde ızgara çizer; varsayılan 10×10 birim.

`lil-gui`

Nesne ve özellik adı alır; türüne göre arayüz üretir (bool için onay kutusu).

```
get visible() {
  return this._visible;
}
set visible(v) {
  this._visible = v;
  this.grid.visible = v;
  this.axes.visible = v;
}
```

- AxisGridHelper: tek “visible” özelliği hem eksenleri hem ızgarayı açıp kapatır (getter/setter).
- depthTest = false: eksenler küre içinde de görünür.
- renderOrder: ızgara 1, eksenler 2.

Hiyerarşi örnekleri: araba ve insan

Araba

```
frame
├─ body
└─ tekerlekler
```

Gövdeyi hareket ettirirseniz tekerlekler de gider. Gövde tekerleklerden bağımsız yaylanacaksa ikisi de bir “şasi” (frame) düğümüne bağlanır.

İnsan

```
insan
├─ gövde, kollar, bacaklar
├─ parmaklar (en az 28)
├─ ayak parmakları (28)
└─ yüz, çene, gözler…
```

Sahne grafiği hızla karmaşıklaşır; bu çizim bile basitleştirilmiş bir örnektir.

Tank örneği: yarı karmaşık bir sahne grafiği

Tankın 6 tekerleği ve bir kulesi (turret) var. Bir yolu izler ve etrafta dönen bir küreyi (hedef) nişan alır.

```
targetOrbit
└─ targetElevation
   └─ targetBob
      ├─ targetMesh
      └─ targetCameraPivot
```

- **targetOrbit:** earthOrbit gibi dönen düğüm.
- **targetElevation:** öteleme ve temel yükseklik.
- **targetBob:** yalnızca yukarı-aşağı sallanır.
- **targetMesh:** kendi etrafında dönen, rengi değişen küp.

Hareket: hedef, tank ve kule

```
// hedef
targetOrbit.rotation.y = time * .27;
targetBob.position.y = Math.sin(time * 2) * 4;

// tank: eğri üzerinde ilerler
curve.getPointAt(tankTime % 1, tankPosition);
curve.getPointAt((tankTime + 0.01) % 1, tankTarget);
tank.position.set(tankPosition.x, 0, tankPosition.y);
tank.lookAt(tankTarget.x, 0, tankTarget.y);

// kule hedefe bakar
targetMesh.getWorldPosition(targetPosition);
turretPivot.lookAt(targetPosition);
```

- SplineCurve: 0.0 başlangıç, 1.0 son.
- Biraz ilerideki nokta, lookAt ile tankın yönünü verir.
- Kule tankın çocuğu olduğu için otomatik taşınır.
- Kuleyi çevirmek için hedefin dünya konumu yeter.

Dört kamera ve dönen tekerlekler

`camera` Bağımsız (detached) kamera.	`turretCamera` Kulenin üzerinde, hedefe bakar.	`targetCamera` Hedefin yakınında, tanka bakar.	`tankCamera` Tankın arkasının üstünde.

```
wheelMeshes.forEach((obj) ⇒ { obj.rotation.x = time * 3; });
const camera = cameras[time * .25 % cameras.length | 0];
```

targetCameraPivot, targetBob'un çocuğudur ve tanka bakar: kameranın hedefin içinde kalmasını önlemek için kamerayı hedeften öteler. Kameralar render anında sırayla değişir.

05 · Sahne grafiği · Sonuç

Object3D düğümleri oluşturmak ve nesneleri onlara bağlamak, three.js'i verimli kullanmanın en önemli adımlarından biridir.

Sahne grafiği olmadan Ay'ın veya tekerleklerin konumunu hesaplamak oldukça karmaşıktır; sahne grafiğiyle son derece kolay olur.

three.js'in yapısı (Architecture)

Three.js, web sayfasına 3B içerik eklemeyi kolaylaştıran bir kütüphanedir. Mimarisi iki temele dayanır:

Renderer

Sahneyi ve kamerayı alır; 2B bir görüntüyü canvas etiketine çizer.

Scene Graph

Ebeveyn–çocuk ağacı: Camera, Mesh, Light ve Group nesnelerini barındırır.

Mesh

Geometry (şekil) + Material (renk, kaplama, parlaklık).

06 · Özet

Temel kurulum ve kullanım (Setup & Usage)

1. ES6 modülleri ve HTML5 canvas etiketiyle başlanır.
2. WebGLRenderer oluşturulup canvas'a bağlanır.
3. PerspectiveCamera: görüş alanı (fov), en-boy oranı, yakın ve uzak düzlem.
4. Sahneye küp (BoxGeometry + MeshBasicMaterial) eklenir, kamera konumu (position.z) ayarlanır, render döngüsü çalıştırılır.

three.js, WebGL'in karmaşıklığını soyutlar: sahne, kamera, geometri, malzeme ve ışık nesnelerini kolayca birleştirip tarayıcıda 3B sahneler kurmanızı sağlar.